

Lab 03 Processos

Introdução

Um processo é uma instância de um programa que está a executar. Um processo pode ter 1 ou mais tarefas.

Revisão

Comece por analisar, compilar e executar o código apresentado em baixo. Analise e compreenda o output produzido.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    printf("Start of the main function...\n");
    int id, i;
    id=fork();

    if(id==0)
    {
        /*child process*/
        printf("\nChild section...\n");
        i=1;
    }
    else if(id>0)
    {
        /*parent process*/
        printf("\nParent section...\n");
        i=0;
        wait(NULL);
    }

    printf("End of the main function...%d\n", i);

    return 0;
}
```

Exercício 1. Pai e filho

Avale o output produzido pela execução do seguinte código. Identifique as relações entre os valores de mypid, getpid e de getppid em cada seção (pai e filho) e responda às questões apresentadas em baixo:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main(void) {
```

```

pid_t mypid;
pid_t childpid;

mypid = getpid();
childpid = fork();

if (childpid == 0) {
    sleep(10);
    printf("child mypid :%ld\n", (long)mypid);
    printf("child getpid() :%ld\n", (long)getpid());
    printf("child getppid() :%ld\n", (long)getppid());
}
else {
    printf("mypid :%ld\n", (long)mypid);
    printf("getpid() :%ld\n", (long)getpid());
    printf("getppid() :%ld\n", (long)getppid());
}

return 0;
}

```

- Porque é que o resultado é diferente sempre que executa o programa?
- Qual o processo que termina primeiro, o pai ou o filho?
- O que acontece a um processo filho em execução quando um processo pai termina? O que acontece ao valor do seu `getppid()`? De quem é o PID? (enquanto o programa executa, abra outro terminal e use o comando `ps` para descobrir a resposta, e.g. `ps -ef`)
- Quem é o pai do processo original?
- O que acontece se adicionar um `sleep(15)` dentro do bloco `else`, assim que o bloco começa?
- Adicione agora o comando `wait(NULL)` depois do `sleep(15)`. Utilize o comando `ps -ef` noutra terminal para verificar o estado dos processos enquanto o `sleep` está a executar. E o que acontece depois de ter terminado? Caso facilite a análise, pode aumentar o tempo do `sleep`.

NOTAS:

Abra outro terminal e utilize o comando `ps -ef` para avaliar adequadamente a hierarquia dos processos e identificar algum processo zombie ou órfão.

Pode sempre recorrer ao manual para aceder à especificação de determinado comando/função (`man comando`)

Exercício 2. Múltiplos processo (fork)

Quantos processos seriam criados ao executar o bloco de código apresentado em baixo? Quantas vezes, e com que valores, é que a variável `i` seria apresentada no ecrã? Crie um ficheiro de código fonte `.c` que implemente o trecho de código apresentado. Execute o código para verificar a sua resposta.

```

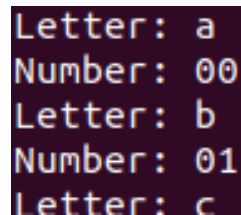
for(int i = 0; i < 3; i++){

    fork();
    printf("%d: %d\n", getpid(), i);
}

```

Exercício 3. Tarefas

Desenvolva um programa que lança 2 tarefas: a 1ª tarefa irá ser responsável por escrever no ecrã o abecedário (uma letra por linha, a cada 1 segundo); a 2ª tarefa será responsável por escrever no ecrã os números de 1 a 26 (um por linha, a cada 1 segundo). O output deve ser o seguinte:



```
Letter: a
Number: 00
Letter: b
Number: 01
Letter: c
```

Depois de lançar as duas tarefas, utilize a instrução `pthread_join()` de maneira a “esperar” que ambas terminem. Compile e execute o programa, para garantir que tem o output esperado.

- Comente as instruções `pthread_join()` que utilizou e adicione, logo de seguida, um `sleep(7)`. Depois deste `sleep`, imprima no ecrã uma mensagem a indicar que “já acordou...”. Compile e execute o programa. O output inicial mantém-se? Porquê?
- O que acontece se, antes da mensagem “acordou...”, adicionarmos a instrução `pthread_exit(NULL)`? Verifique que processos estão em execução nos vários momentos.

Exercício 4. Tarefas

Pretende-se desenvolver um programa que lança 2 tarefas para incrementar um **contador partilhado**, desde 0 a 1.000.000. Cada tarefa será responsável por incrementar o contador 500.000 vezes (i.e., se o contador iniciar a zero, no final das duas tarefas terminarem a execução, o valor final do contador deverá ser 1.000.000).

Na função `main`, deverá começar por imprimir o valor inicial do contador (que será zero) e deverá lançar as 2 tarefas. Depois da função `main` esperar que as duas tarefas terminem a sua execução, volta a imprimir o valor do contador e o programa termina.

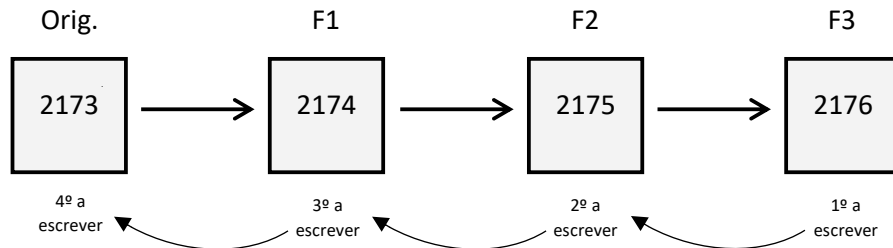
Compile e execute o programa, para verificar o output.

- Experimente correr o programa várias vezes. O output é o esperado a cada execução? Porquê?
- Que mecanismo teria de ser implementado para que o programa tivesse o comportamento esperado? Justifique.
- Faça as alterações necessárias para forçar as tarefas a correr sequencialmente (i.e., a tarefa 2 só pode iniciar a execução depois de ter a certeza que a tarefa 1 já terminou). O comportamento alterou-se em comparação com o comportamento da alínea a)? Justifique.

Exercício 5. Impressão inversa

Desenvolva um programa que crie uma sequência de processos onde cada processo imprime o seu próprio PID e o PID do seu pai (PPID). No output do programa, a informação deve ser apresentada em ordem inversa ao da criação dos processos, como ilustrado de seguida:

```
How many processes you want to create?
3
I am 2176 and my father is 2175
I am 2175 and my father is 2174
I am 2174 and my father is 2173
I'm the Original Process and my PID is:2173
```



TIP: O processo original poderá começar por criar um processo filho e ficar à espera que este termine (`wait`). Só quando este filho terminar é que vai imprimir uma frase a dizer que é o processo original e indicar o seu PID.

Por outro lado, esse filho do processo original poderá, com recurso a um ciclo de repetição, lançar N processos, de maneira que, na secção destinada aos pais, fiquem também estes à espera que os seus filhos terminem. Depois de esperarem (i.e., depois de serem assinalados que os seus filhos terminaram), podem então indicar o seu PID e PPID.

Exercício 6. Invocação de Programas – funções `exec()`

Desenvolva o código de um programa, `prog1.c`, que comece por imprimir no ecrã a frase “[PID] prog1 starting...”. De seguida, deverá fazer `sleep` durante 5 segundos e finalmente imprimir no ecrã a frase “[PID] prog1 finished and exiting...”. Compile e execute.

Desenvolva um 2º programa, `prog2.c`, que deverá apenas imprimir no ecrã a frase “[PID] prog2 was launched...”. Compile e execute.

Faça as alterações necessárias no `prog1` para que este invoque o `prog2` logo após imprimir a primeira frase (dica: `execlp("./nomeprog2", "nomeprog2", NULL)`). Volte a compilar o `prog1` e execute-o. Analise o output produzido.

- O output é o esperado? Porque é que `prog2` e `prog1` indicam o mesmo PID?
- O programa `prog1` chega a terminar? O que acontece a `prog1` ao invocar outro programa?